core library source (`libgcs.c`) and its export header (`libgcs.h`)

Technical Module Description: Core API Gateway (`libgcs.c` / `libgcs.h`)

1. Overview

The `libgcs` core implementation (`libgcs.c`) and its exported interface (`libgcs.h`) act as the **primary API Gateway** for the GCSORT library. This module exposes public entry points prefixed with `gcs_` to control, configure, and execute sorting, merging, and record refactoring actions. It serves as an abstraction layer designed to facilitate smooth interoperability between native C applications and GnuCOBOL standalone programs or callers.

2. Key Architectural Features

2.1 Context-Aware GnuCOBOL Runtime Binding

A critical architecture feature of this module is its ability to identify the hosting context dynamically through `gcs_check_cobol_runtime()`.

- **Dual-Caller Identification:** It detects whether the parent program calling the library is written in native COBOL (`GCS_CALLER_CBL`) or standard C (`GCS_CALLER_C`).
- **Cross-Platform Dynamic Linking:** On Windows, it systematically scans and binds to standard runtime variations (`libcob-*.dll` or `libcob.dll`). On Linux, it probes the active global namespace using `dlsym(RTLD_DEFAULT, ...)` before attempting explicit programmatic hooks via `dlopen()`.
- **Decoupled Initialization:** If a pure C environment is detected, it handles background runtime spin-ups using `cob_init_nomain()`, avoiding duplicate framework instantiation errors if a COBOL runtime is already active.

2.2 Framework Simulation and Environment Mocking

When executing inside non-COBOL processes, the library mocks a native GnuCOBOL context inside `gcs_init_cob()` to fulfill dependencies expected by internal file operations:

- Registers an internal proxy module (`cob_module_enter`) and initializes essential runtime global metadata pointers (`gcs_cob_glob_ptr`).
- Mocks vital runtime control flags, local parameters, and environmental characteristics including decimal positions, international currency characters (`$`), and call structures.

- Establishes local call-frame execution stacks (`frame_stack`) and hooks runtime program cleanup procedures with `cob_set_cancel`.

2.3 Operational State and Lifecycle Management

The library controls operational state through a unified job descriptor tracker object (`struct job_t`):

- **`gcs_sort_init()`**: Acts as the master constructor. It instantiates the job tracking structure, allocates internal key buffers, and sets up the foundational GnuCOBOL context.
- **`gcs_sort_run()`**: Orchestrates the multi-stage file execution pipeline. It parses active configuration paths, populates internal memory lists via `gcs_sort_LoadFile()`, runs sorting configurations, handles disk segmentation overflows, and pipes rows to destination files.
- **`gcs_sort_cleanup()`**: Safely destroys data models, clears memory heaps, and guarantees orderly framework wrap-ups by executing `cob_tidy()` if a pure C host initialized the background runtime.

2.4 External API Proxies & Metrics

The module exposes streamlined parameter routing paths to safely update data structure configurations:

- **Record Refactoring Filters:** Exposes safe boundaries to bind preprocessing (`gcs_inrec_*`) and postprocessing (`gcs_outrec_*`) layouts to rewrite and pad records.
- **File Layout Mappings:** Relays characteristics like sequential data organization types, physical maximum bounds, and character sets (ASCII/EBCDIC) directly to the GCSORT back-end tracking mechanisms.
- **Operational Diagnostic Data:** Provides explicit functions to access global performance tracking rows, outputting high-precision row counters (`gcs_stats_rec_*`) for standard processing pipelines, exclusions, and file distribution balances.

---

3. Interface Compilation Matrix (`libgcs.h`)

The export header features a clean compilation design to enforce clean linking behavior across compilation targets:

- **Unified Export Macros:** Employs explicit conditional definition branches (`GCS_API`) mapping to `__declspec(dllexport)` on Windows targets and `__attribute__((visibility("default")))` on GCC environments.
- **Version Compatibility Safeguards:** Integrates feature guards ensuring compatibility across GnuCOBOL versions. For GnuCOBOL releases greater than or equal to v3.2.0, it routes custom collating behaviors via native engine methods; on legacy frameworks, it falls back gracefully via empty preprocessing definition mappings.

4. API Execution Sequence and Lifecycle

To correctly execute sorting or merging operations using **libgcs**, calling applications (whether written in native C or interfacing via wrapper layers) must adhere to a strict **Order of Operations**. The library lifecycle is divided into four distinct phases: **Initialization, Configuration**, **Execution**, and **Cleanup**.

4.1 Lifecycle Flow Diagram

```
[Phase 1: Init]       gcs_sort_init()
                             │
                             ▼
                      gcs_colseq() / gcs_cb_sign()
                             │
[Phase 2: Config]            ▼
                      gcs_sort_init_key()
                             │
                             ▼
                      gcs_sort_use() / gcs_sort_give()
                             │
                             ▼
                      gcs_inrec_*() / gcs_outrec_*() [Optional]
                             │
                             ▼
                      gcs_include_*() / gcs_omit_*() [Optional]
                             │
                             ▼
[Phase 3: Execute]    gcs_sort_run()
                             │
                             ▼
[Phase 4: Cleanup]    gcs_sort_cleanup()
```

4.2 Step-by-Step Sequence Description

Phase 1: **Initialization**

1. **gcs_sort_init()**

Purpose:This **must** be the very first function called.

Internal Action: It instantiates the global job structure (
struct job_t), allocates system memory for sorting structures, and triggers **gcs_init_cob()**

* **Context Handling**:

gcs_init_cob() dynamically detects the host environment. If it runs under a pure C context, it programmatically bootstraps the GnuCOBOL runtime using **cob_init_nomain()**
and builds the necessary mock framework structures (frame stacks, module descriptors).

Phase 2: Configuration

2. Global Environment Setup (Optional)

* **Functions**: **gcs_colseq(int ns)**and **gcs_cb_sign(int ns)**.

* **Purpose**: Defines the character set sorting weights (Native, ASCII, or EBCDIC) and specifies numeric sign representations. If custom conversion tables are needed,**gcs_ebcdic_table()** is invoked here.

3. **Operation Type Definition**

Function: gcs_sort_set_oper(char c)

Purpose: Instructs the back-end engine whether to execute a standard Sort operation ('S') or a Merge operation ('M') or Copy operation ('C').

4. Sort Key Specification

Function: **gcs_sort_init_key(int position, int length, const char* ctype, const char* cdirection)**

Purpose: Defines the sorting criteria. For composite multi-key sorting, this function is called **multiple times in chronological sequence**, matching the desired priority layout (Primary Key, Secondary Key, etc.).

5. File Interoperability Mapping

Functions:**gcs_sort_use()**(Input) and **gcs_sort_give()**(Output).

Purpose: Binds physical file paths to the engine. It maps critical file meta-attributes—such as record organization types (e.g., Sequential), record formats (Fixed/Variable), and

minimum/maximum record lengths—into the active context tracking arrays. For a `MERGE` operation, **gcs_sort_use()** must be called multiple times (once per input file).

6. Record Refactoring Filters (Optional)

Functions: **gcs_inrec_*() or unset gcs_outrec_*()**

Purpose: Configures structural data refactoring (constant injections, byte-range movements, or byte paddings). These functions map record modification rules applied either before sorting occurs (INREC) or immediately prior to writing output blocks (OUTREC).

7. Record Selection Filters (Optional)

Functions: gcs_cond_*(), gcs_addInclude(), gcs_addOmit()

Purpose: Configures conditional criteria to filter input file records based on field-to-field or field-to-value comparisons. These functions dynamically build logical AND/OR condition trees to either explicitly include or omit matching records (mutually exclusive) during the file reading phase.

Phase 3: Execution

8. gcs_sort_run()

Purpose: Executes the actual processing pipeline.

Internal Action: It automatically coordinates sub-modules to review file descriptors, streams rows into memory buffers via
gcs_sort_LoadFile(), triggers the quicksort/mergesort sorting thread pools, handles intermediate disk segmentation on buffer overflows, and flushes organized blocks into the configured target output file. It returns `0` on success or an error code on failure.

Phase 4: Cleanup

9. Diagnostic Metrics Gathering (Optional)

Functions: **gcs_stats_rec_total()**, **gcs_stats_rec_writeout()**, etc.

Purpose: Queries high-precision internal diagnostic counters to generate processing metrics and verify total records manipulated, filtered, or discarded.

10. gcs_sort_cleanup()

Purpose: Safely terminates the operational instance.

Internal Action: Deallocates global heap items, unlinks file descriptors, and safely flushes remaining file streams.

4.3 C Code Implementation Example

```
#include <stdio.h>
#include <libgcs.h>

int main() {
 int return_code = 0;

 // 1. Initialize the library and the underlying COBOL environment context
      if (gcs_sort_init() != 0) {
      fprintf(stderr, "Failed to initialize GCSORT environment.\n");
      return 1;
 }

 // 2. Configure global engine behavior
      gcs_colseq(1);             // Force ASCII collating sequence
       gcs_cb_sign(0);            // Standard ASCII sign convention
       gcs_sort_set_oper('S');    // Configure as an explicit SORT operation

 // 3. Define sorting keys (Primary: Pos 1 Len 3 Descending; Secondary: Pos 4
Len 5 Ascending)

    gcs_sort_init_key(1, 3, "ZD", "D");
    gcs_sort_init_key(4, 5, "ZD", "A");

    // 4. Map the dataset files into the back-end tracker
    gcs_sort_use("../files/input.dat", "SQ", "V", 20, 90);
    gcs_sort_give("../files/output.dat", "SQ", "V", 20, 90);

    // 5. Execute the processing pipeline
    printf("Executing GCSORT sorting mechanism...\n");
    return_code = gcs_sort_run();
    printf("GCSORT execution complete. Engine Return Code: %d\n", return_code);

    // 6. Gather processing statistics
    printf("Total records handled: %lld\n", (long long)gcs_stats_rec_total());
    printf("Total records written: %lld\n", (long
long)gcs_stats_rec_writeout());

    // 7. Perform mandatory runtime heap cleanup
    gcs_sort_cleanup();

    return return_code;
}
```